



UNIVERSIDAD DE ANTIOQUIA

Facultad de Ingeniería

Informática 2: Unidad 3 – Taller Evaluativo

Introducción a la informática médica.

Monitor: Juan Esteban Pineda Lopera
jesteban.pineda1@udea.edu.co

1 Objetivo.

Desarrollar una aplicación en Python para automatizar la lectura, extracción y almacenamiento de metadatos de archivos DICOM, así como realizar procesamiento básico de imágenes médicas usando OpenCV, aplicando los estándares de la informática médica y herramientas de software libre.

2 Introducción y contexto.

En la informática médica, los sistemas de comunicación y archivado de imágenes (PACS) son fundamentales. Estos sistemas se comunican utilizando el estándar DICOM para garantizar la interoperabilidad. En este taller, se simulará una parte de este flujo: Se darán un conjunto de estudios médicos en formato DICOM, y la tarea será procesarlos para hacer su información accesible y consultable, como lo haría un software real tipo dcm4chee.

Se utilizará la librería `pydicom` para interactuar con los archivos, `OpenCV` para el procesamiento de imágenes, y se almacenarán los resultados en una base de datos, demostrando una aplicación práctica de bioingeniería en el manejo de datos clínicos.

3 Requerimientos e instrucciones.

A continuación se describen los requisitos evaluativos del trabajo. Este trabajo debe entregarse como un repositorio en *GitHub*. Puede realizarse individualmente o en parejas. Debe dejarse claro en el archivo `README.md` el nombre de los integrantes del equipo.

3.1 Preparación del entorno

- Crear un entorno virtual de *Python* para el proyecto.
- Instalar las librerías necesarias, incluyendo *pydicom*, *numpy*, *pandas* y *opencv-python*.
- Descargar un conjunto de archivos *DICOM* de prueba. Puede usar los que provee la librería *pydicom* en su **repositorio** (en el directorio `data_store/data`), o cualquier otra base de datos que encuentre libremente.

4 Desarrollo de la aplicación.

Su aplicación deberá ser capaz de ejecutar los siguientes pasos de forma organizada. Se recomienda el uso de Programación Orientada a Objetos (POO) para encapsular la lógica, por ejemplo, creando una clase `ProcesadorDICOM`.

4.1 Carga de archivos DICOM.

- Implementar una función o método que reciba la ruta de un directorio.
- Este método debe escanear el directorio, identificar todos los archivos *DICOM* y cargarlos utilizando `pydicom.dcmread()`. Debe asegurarse de manejar posibles errores si un archivo no es un *DICOM* válido.

4.2 Extracción de metadatos.

Para cada archivo *DICOM* cargado, extraer la siguiente información. Investigar el nombre del *tag* correspondiente en el estándar *DICOM*:

- Identificador del paciente.
- Nombre del paciente.
- Identificador único del estudio.
- Descripción del estudio.
- Fecha del estudio.
- Modalidad de la imagen (CT, MR...).
- Número de filas de la imagen.
- Número de columnas de la imagen.

Nota: Es muy posible que alguno(s) de los *tags* no estén disponibles debido a procesos de anonimizado; debe manejar estos casos donde algún *tag* no esté presente en algún archivo.

4.3 Estructuración de los datos.

Almacenar los metadatos extraídos de todos los archivos en un **Dataframe de Pandas**. Cada fila debe representar un archivo *DICOM* y cada columna uno de los *tags* extraídos.

4.4 Análisis de imagen con NumPy.

Para cada imagen *DICOM* (accesible en el atributo `.pixel_array`), realizar un cálculo simple usando la librería `numpy` del valor de intensidad promedio de los píxeles de cada imagen. Añadir este valor como una nueva columna llamada **IntensidadPromedio** en el **Dataframe** creado anteriormente.

4.5 Procesamiento de imágenes con OpenCV.

Esta sección introduce el uso de `OpenCV` (`cv2`) como herramienta de procesamiento de imágenes médicas. Dado que los arreglos de píxeles obtenidos con `.pixel_array` son compatibles con `numpy`, pueden utilizarse directamente con `OpenCV` tras una normalización adecuada.

Para cada imagen *DICOM*, realizar los siguientes pasos:

1. **Normalización:** Convertir el arreglo de píxeles a un formato de 8 bits (`uint8`) escalando los valores al rango `[0, 255]`. Esto es necesario porque las imágenes *DICOM* suelen tener profundidades de 12 o 16 bits.
2. **Ecualización del histograma:** Aplicar la función `cv2.equalizeHist()` sobre la imagen normalizada. Esta técnica mejora el contraste redistribuyendo la intensidad de los píxeles, y es ampliamente usada como paso de preprocesamiento en imágenes médicas.
3. **Detección de bordes:** Aplicar el detector de bordes de Canny (`cv2.Canny()`) sobre la imagen ecualizada, usando umbrales que el estudiante debe justificar. Los bordes detectados resaltan estructuras anatómicas relevantes.
4. **Guardado de resultados:** Guardar las imágenes procesadas (ecualizada y con bordes) en un directorio de salida como archivos `.png`, usando `cv2.imwrite()`. El nombre de cada archivo debe incluir un identificador del estudio o del archivo de origen.

Nota: No todos los archivos *DICOM* contienen datos de píxeles accesibles (algunos son de modalidad *SR*, *PR* u otros no-imagen). Debe manejar estos casos con un bloque `try/except` adecuado.

4.6 Informe y discusión.

En el repositorio, además del código, debe ir adjunto el `README.md` que incluya de manera organizada:

1. Breve descripción del proyecto.
2. Explica brevemente por qué *DICOM* y *HL7* son cruciales para la interoperabilidad en salud y en qué se diferencian conceptualmente.
3. ¿Qué ventajas y limitaciones tiene aplicar la ecualización de histograma y la detección de bordes con Canny en imágenes médicas como radiografías o tomografías? ¿En qué escenarios clínicos o de preprocesamiento podría ser útil o perjudicial su uso?
4. Mencionar dificultades encontradas y la importancia de las herramientas de *Python* para el análisis de datos médicos.

5 Rúbrica de evaluación.

Table 1: Tabla de Criterios de Evaluación

Criterio de Evaluación	Descripción	Porcentaje
1. Funcionalidad del Código	El script se ejecuta sin errores y cumple con todos los pasos solicitados (carga, extracción, análisis NumPy y procesamiento OpenCV).	35%
2. Calidad del Código y POO	El código está bien estructurado, es legible y utiliza correctamente funciones y/o clases para organizar la lógica.	20%
3. Correcta Extracción de Metadatos	El programa extrae correctamente todos los tags DICOM solicitados y los organiza en un DataFrame de Pandas.	15%
4. Procesamiento con OpenCV	Se implementan correctamente la normalización, ecualización del histograma, detección de bordes con Canny y el guardado de imágenes resultantes.	15%
5. Informe y Discusión (README.md)	El informe responde a todas las preguntas teóricas, incluyendo la reflexión sobre OpenCV en contextos médicos, demostrando comprensión de los conceptos de estándares y software libre.	15%
Total		100%